

UM EXEMPLO PRÁTICO

Apresentar para uma sala que não partilha um único idioma.

Aprender português a sério é a verdadeira resposta, e nisso sou mais lento do que gostaria. Entretanto existe uma resposta de engenharia: dar à sala legendas no idioma dela, e dar-me as perguntas no meu. Esta apresentação é sobre **como** isto foi construído — mais do que sobre o que faz.

Legendas ao vivo para reuniões · feito em dias, não meses · custo zero para funcionar

Escrever funciona. Falar é a parte difícil.

- A comunicação escrita entre programadores é em inglês, e isso funciona.
- A fala ao vivo é **onde falha** — a começar por mim. O meu português ainda está em construção, por isso numa reunião perco as perguntas, não os diapositivos.
- E não sou só eu: inglês à velocidade da fala não é igualmente confortável para toda a gente na sala.
- O Teams tem legendas ao vivo. Legendas **traduzidas** exigem Premium, e toda a gente precisaria dele.

A forma do problema

Uma apresentação é uma emissão **um-para-muitos** com uma cauda de perguntas **muitos-para-um**. Essas duas direções têm requisitos completamente diferentes — e isso acabou por ser a coisa mais importante a reparar.

Eu → sala	latência crítica
-----------	------------------

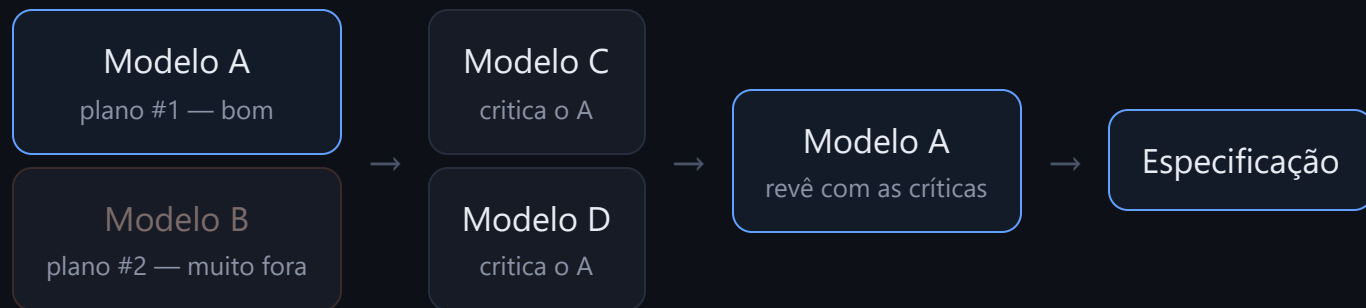
Sala → eu	exatidão crítica
-----------	------------------

A mesma ideia, com dois anos de diferença

	ANTES	AGORA
Reconhecimento de fala	Serviço pago na nuvem, chave de API, faturação à hora	Integrado no navegador. Grátis, sem limite.
Tradução	API paga, ida e volta pela rede a cada frase	Corre no dispositivo . Sem rede, sem chave, sem quota.
Deteção de idioma	Modelo especializado, afinação própria, falhas próprias	Uma frase no prompt.
Construí-lo	Semanas de uma equipa. Nunca seria aprovado para o problema de uma pessoa.	Dias de uma pessoa. Sem precisar de aprovação.

A mudança interessante é a última linha. **A economia mudou antes da tecnologia.** Problemas que afetam uma pessoa nunca justificavam um projeto. Agora justificam uma tarde — e por isso são resolvidos.

Não perguntei a um modelo. Convoquei um conselho.



- Fazer a **mesma pergunta a 2–4 modelos**, em separado. Barato e paralelo.
- Escolher o melhor plano. Aqui o primeiro já era bom e o segundo estava claramente errado — **o valor está nessa comparação**, não na média.
- Entregar o vencedor aos **outros como críticos**, não como autores.
- Devolver as críticas **ao autor original**.

Isto já tem nome

O **LLM Council** de Karpathy — consulta paralela, revisão por pares, síntese. A variante que usei é o **conselho adversarial**: os revisores são instruídos a encontrar falhas, não a concordar.

Porque funciona: o ponto cego de um único modelo torna-se a resposta final. Um crítico com outro ponto cego não partilha esse ponto cego.

O que os críticos mudaram de facto

O PRIMEIRO PLANO DIZIA

O FINAL DIZ

PORQUE MUDOU

Azure Speech como motor

Nenhum serviço pago

Exige cartão mesmo no plano gratuito. Mata a adoção pelos colegas.

Aceitar PPTX e converter com o LibreOffice

Apenas PDF

Eliminou um servidor, um analisador de OOXML e uma classe inteira de erros de layout.

Reamostrador de áudio à mão + teste unitário

Um argumento do construtor

O navegador já reamostra. Um dia de trabalho que não era preciso fazer.

Idiomas fixados no código

Um objeto de perfil

«Os teus colegas têm outros idiomas.» Não tinha pensado nisso de todo.

Todas estas mudanças **retiraram** trabalho. Os críticos não acrescentaram funcionalidades — apagaram planos. É a coisa de maior valor que um revisor pode fazer, e é exatamente o que um único modelo entusiasmado não faz a si próprio.

A especificação é o artefacto. O código vem a jusante.

- Escrevi e reescrevi uma **especificação**, não prompts. Requisitos, critérios de aceitação, não-objetivos explícitos.
- Cada reversão do diapositivo anterior está **registada nela com o motivo**. Daqui a seis semanas não volto a discutir o Azure.
- A indústria chama-lhe **spec-driven development**, e 2026 foi o ano em que todas as ferramentas de agentes lançaram uma versão disto.
- O motivo apontado bate certo com a minha experiência: os agentes escrevem bem código e **adivinham mal a intenção**. O desvio é o modo de falha, não a sintaxe.

O que fez a diferença

Não-objetivos. «Sem PPTX. Sem contas. Sem servidor.» Escritos, deixam de voltar.

CrITÉRIOS de aceitação como testes. «As transcrições não contêm nenhuma frase no outro idioma» é verificável. «Boa qualidade» não é.

Um registo de decisões. A especificação acaba com uma tabela do que foi revertido e porquê — a página mais relida.

Medir antes de construir. Uma página de código.

Antes de escrever a aplicação escrevi um único ficheiro HTML que testava cada pressuposto no portátil real. Demorou minutos e mudou a arquitetura.

147 ms

Reconhecimento de fala do navegador, do fim da frase ao texto final. Eu tinha orçamentado **1000 ms**.

~600

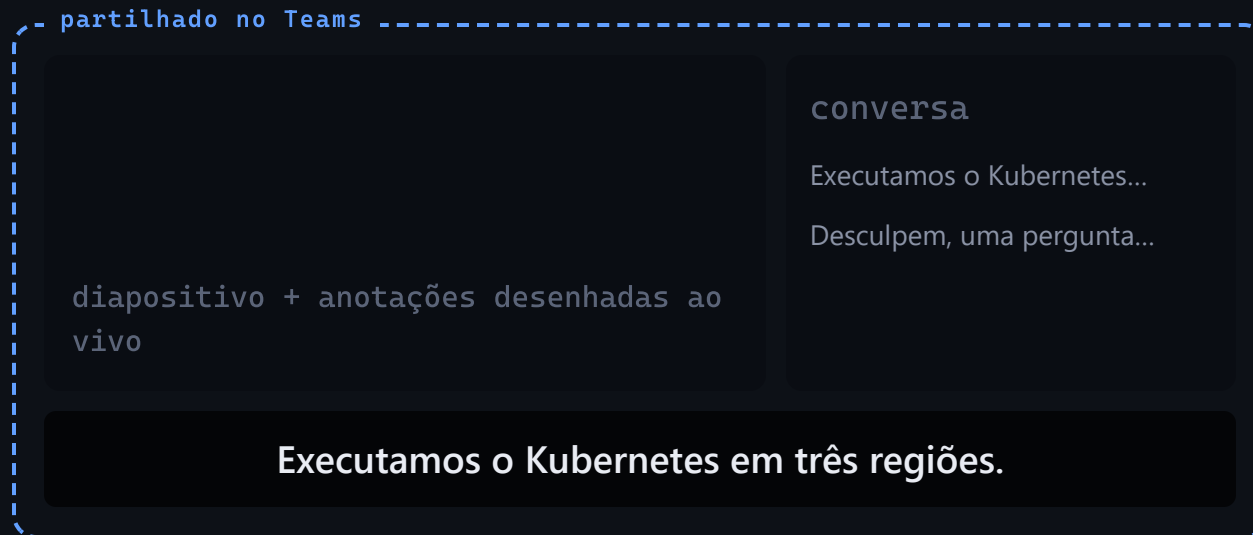
Pedidos a um modelo na nuvem para uma apresentação de 2 horas, ao meu ritmo real de fala. Quase de certeza acima da quota diária gratuita.

0

Drivers, servidores e contas necessários — os três riscos para os quais eu tinha planeado mitigações afinal não existiam.

Os dois números apontavam no mesmo sentido e **inverteram a minha opção por omissão**. Eu tinha planeado usar o modelo na nuvem para a minha própria fala. A medição disse: usar o navegador e guardar a nuvem para a audiência. Vinte vezes mais rápido, e grátis.

Uma janela. A sala acompanha a leitura.



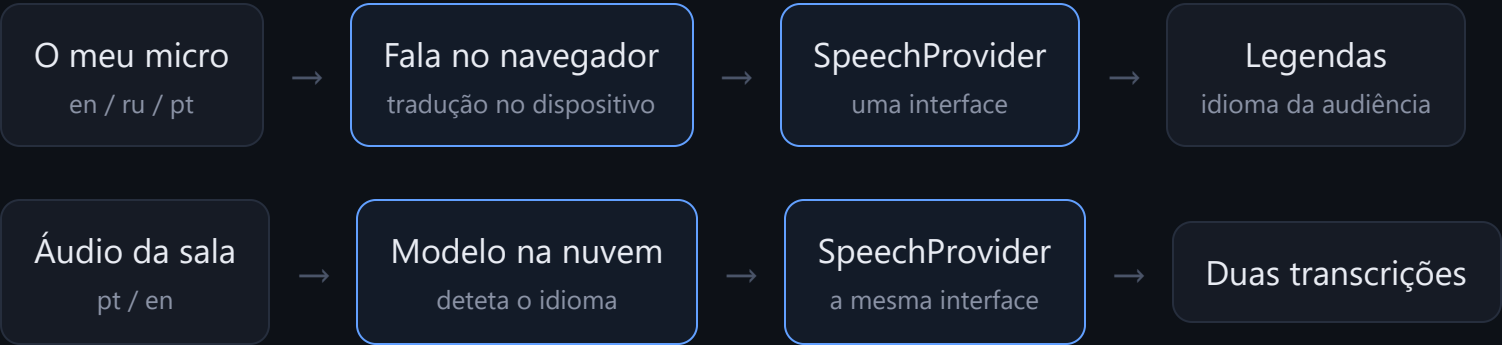
Este diapositivo dizia «duas janelas»

O primeiro desenho tinha uma segunda janela privada: o registo era meu, a sala via apenas o diapositivo.

Depois veio a pergunta óbvia — **para quem é o registo?** É para as pessoas que precisam de ler o que acabei de dizer. Assim, a janela privada ficou sem nada para guardar, e foi apagada.

Está tudo numa única imagem partilhada: diapositivo, legendas, conversa. **Sem bot, sem Premium, nada para instalar.**

Dois canais, dois motores, um contrato



CAMADA	ESCOLHA	PORQUÊ ESSA
Diapositivos	pdf.js em canvas	Única forma fiável de desenhar um diapositivo fiel à impressão
Estado	Uma store Zustand	Diapositivos, registo e anotações num só sítio — sem passar props em cadeia
Anotações	SVG sobre canvas	O PDF nunca é alterado. O navegador trata da deteção de cliques.
Armazenamento	IndexedDB + localStorage	O registo sobrevive a um F5 acidental; nada sai da máquina

Deixar o utilizador escolher, e deixar a escolha selecionar o motor

	FIXO — EU DIGO QUAL O IDIOMA	AUTO — O MODELO DETETA-O
Atraso da legenda	~0.15 s	~3 s
Texto enquanto falo	sim	não — a frase inteira de uma vez
Limites diários	nenhum	quota partilhada
Chave de API	não é precisa	obrigatória
Tenho de me lembrar	premir o chip do canal antes de mudar	de nada

Micro fixo, sala em auto. Eu sei em que idioma vou falar a seguir. Não posso saber em que idioma vem a próxima pergunta.

A definição não é cosmética — ela **seleciona o motor**. A aplicação nunca pergunta «qual o fornecedor?»; faz uma pergunta a que o utilizador consegue responder, e deduz o resto.

Que técnica da moda, e quando

TÉCNICA	O QUE É NA REALIDADE	VALE A PENA QUANDO	É PESO MORTO QUANDO
✓ Conselho de LLMs	Vários modelos respondem e criticam-se uns aos outros	Arquitetura e escolhas irreversíveis	Código de rotina que basta ler
✓ Spec-driven	Uma especificação versionada é a fonte da verdade; o código vem dela	Tudo o que ainda vai mudar no mês seguinte	Um script que se corre uma vez
✓ Bancada de desenvolvimento	Fornecedor simulado + uma rota que repete um guião gravado	A entrada é uma pessoa ao vivo, hardware ou uma API paga	Funções puras — basta chamá-las
✓ Sondar primeiro	Página descartável que mede pressupostos na máquina real	Está prestes a desenhar em torno de um número adivinhado	O número está documentado e é estável
• Avaliações	Conjunto de testes pontuado que apanha regressões	A saída do modelo é o produto e é lançada repetidamente	Ferramenta pontual, com alguém no ciclo em cada execução
• Assistente de voz	Sessão fala-para-fala sobre WebRTC, abaixo do segundo	Mãos e olhos ocupados — a conduzir, a apresentar, a operar	Escrever é mais rápido e silencioso — quase sempre

✓ marca o que este projeto usou de facto. O fornecedor simulado é o que eu defenderia com mais força: **sem ele, cada alteração de interface exige alguém a falar ao microfone.**

Orquestração de agentes: não a usei

Os padrões de 2026 já estão assentes — supervisor, pipeline, distribuição paralela, encaminhador, hierárquico, avaliador-otimizador. Não usei nenhum deles, e penso que foi acertado.

- **Porque não:** uma aplicação, um programador, trabalho quase todo sequencial. A orquestração compra paralelismo que eu não tinha para gastar.
- **Quando compensa:** uma alteração mecânica em muitos ficheiros — migrações, auditorias, varrimentos. Distribuição sob um supervisor.
- **Quando compensará aqui, mais tarde:** a fase de revisão. Vários críticos em dimensões diferentes, a correr ao mesmo tempo, é exatamente o meu conselho — só que automatizado em vez de copiado e colado.

O que melhora à medida que os modelos melhoram

- A tradução no dispositivo melhora → o caminho gratuito deixa de ser o compromisso e passa a ser a opção por omissão.
- Os modelos de fala em streaming amadurecem → deteção e baixa latência deixam de ser um compromisso, e a minha decisão central de desenho **evapora-se**.
- Tokens de áudio mais baratos → o argumento da quota desaparece.
- Contexto mais longo → a reunião inteira resumida, não apenas transcrita.

A interface de fornecedor existe para que **nada disto obrigue a reescrever a aplicação** — basta acrescentar um ficheiro.